

Cloud Computing (cloud)

Storage



Example Exam-Questions

1. Multiple Choice:
 - Fix defined scenario e.g. “map the following technologies to the layers IaaS/PaaS/SaaS-ILayers”.
+ Points for right answers / - Points for wrong answers / not less than 0 points in this question possible
2. Comprehensive/Knowledge Questions:
 - What is the purpose of the kube-apiserver/etcd/kubelet? What happens if it is not available any more?
Might cover lecture, tutorial and chapters from reading list
3. Sourcecode:
 - Given the following log outputs: What can be the failure of the system (e.g. not synced nodes within Ceph)
 - Given following Dockerfile, annotate the different layers and describe what they do.
Might focus more on tutorials.
4. Graphical Questions:
 - Annotate the following (empty) components describing a control plane in K8s.

MSP, Date and Infos

19.12.25: 8:45-9:45h

– 6.0D13

Materials:

- MSP-Summary.docx from Teams
 - At most 1 page per session / week
 - Will be provided by us in a printed form
 - Feel free to collaborate on the content, organize as you want (using the provided teams channel, etc.)
 - Deadline for writing to the document: 7.7.25, 11:00am
- No further materials are allowed

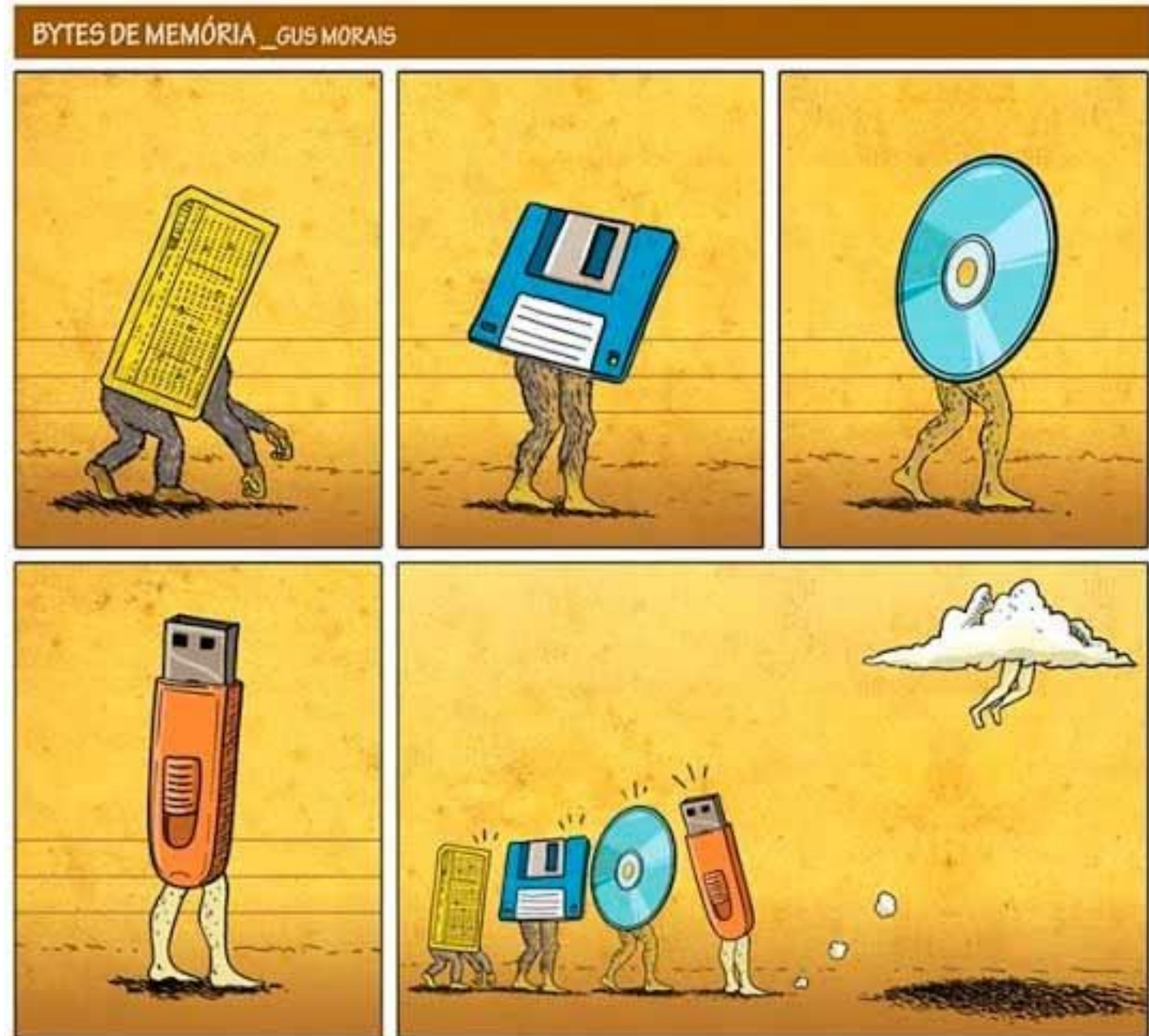
Platform:

- Moodle with LearnStick / SafeExam Browser
 - No Booting, just SafeExam Browser
- Test Exam → 12.12.25
 - You are responsible to make sure your infrastructure works with the setup, this is your chance to check it.
 - Check is not able to be performed remotely.

Custom Resources

- <https://learning.oreilly.com/playlists/43827098-7a87-435e-823e-736586b5694c>
10-storage
- Several Blogposts
 - CERN:
<https://indico.cern.ch/event/649159/contributions/2761965/attachments/1544385/2423339/hroussea-storage-at-CERN.pdf>
- Ceph-Documentation: <https://docs.ceph.com/>
- Kubernetes-Documentation: <https://kubernetes.io/docs/concepts/storage/persistent-volumes/#volume-mode>

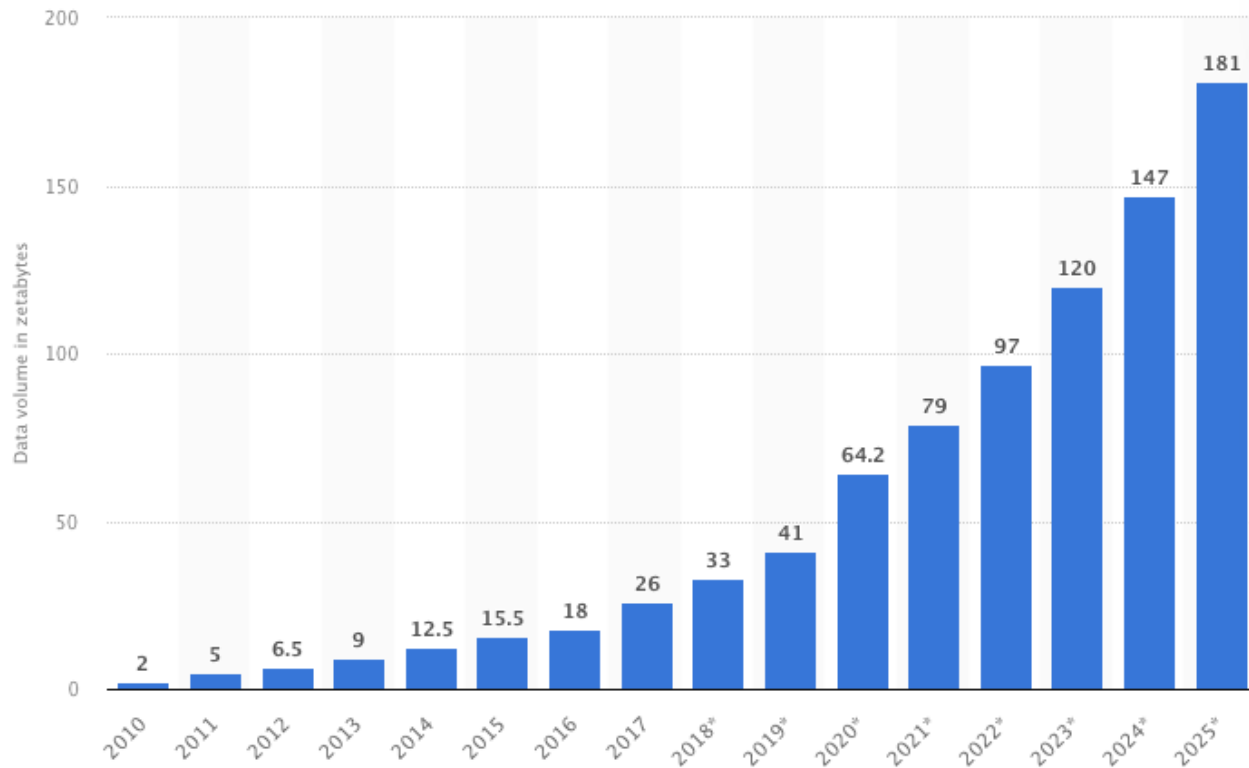
Evolution of Storage



Why do we need Storage?

Just reflect the immense amount:

- 282 pages → 1MB
- 536 books → 1GB
- 549755 book → 1TB
- 192 Library of Congress → 1PB
- 1000000 PB → 1 ZB



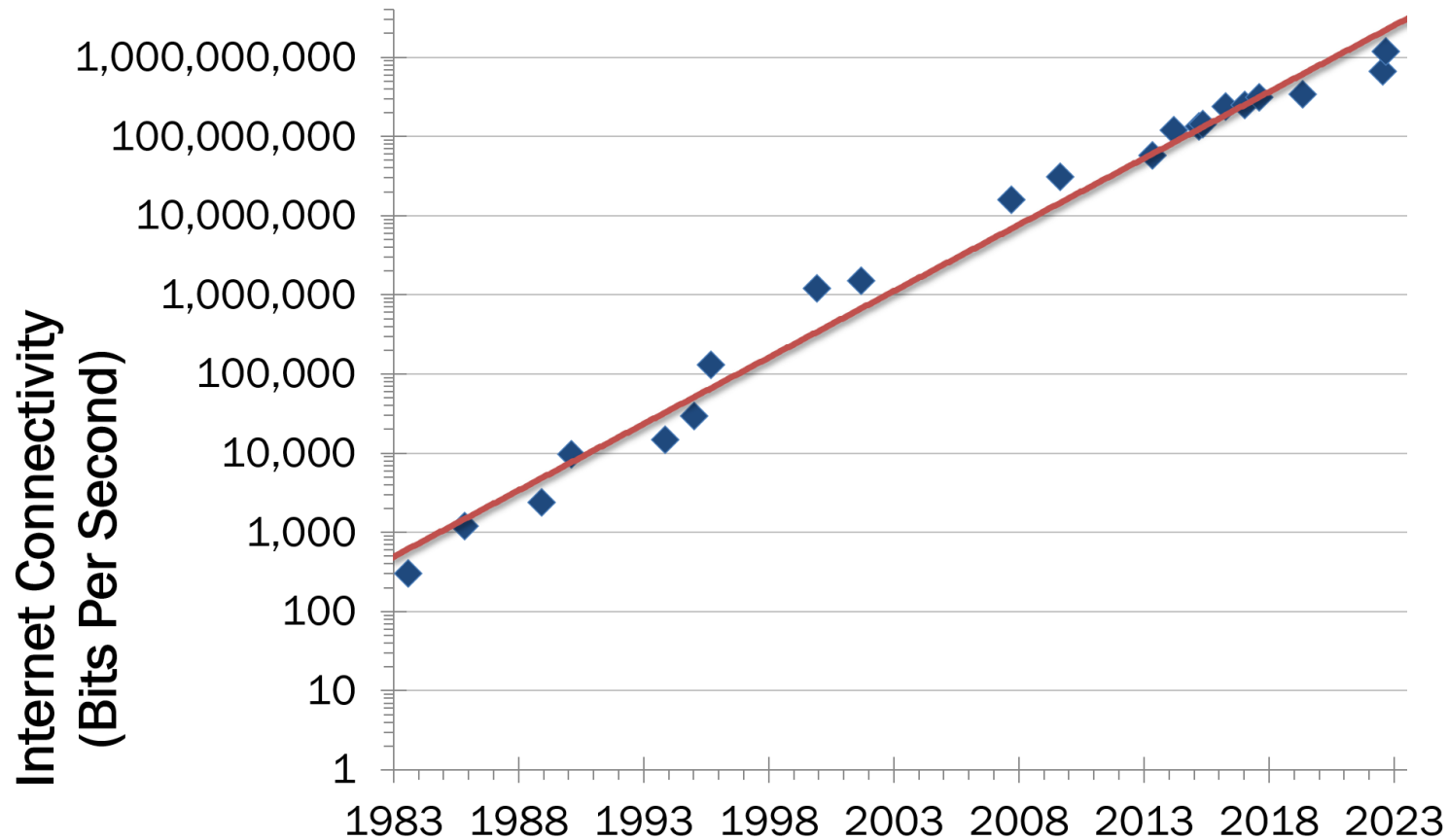
© Statista 2023

Show source

Additional Information

Nielsen's Gesetz: "High-end users connection speed increases by 50% per year."

Why Storage matters? → Bandwidth increases

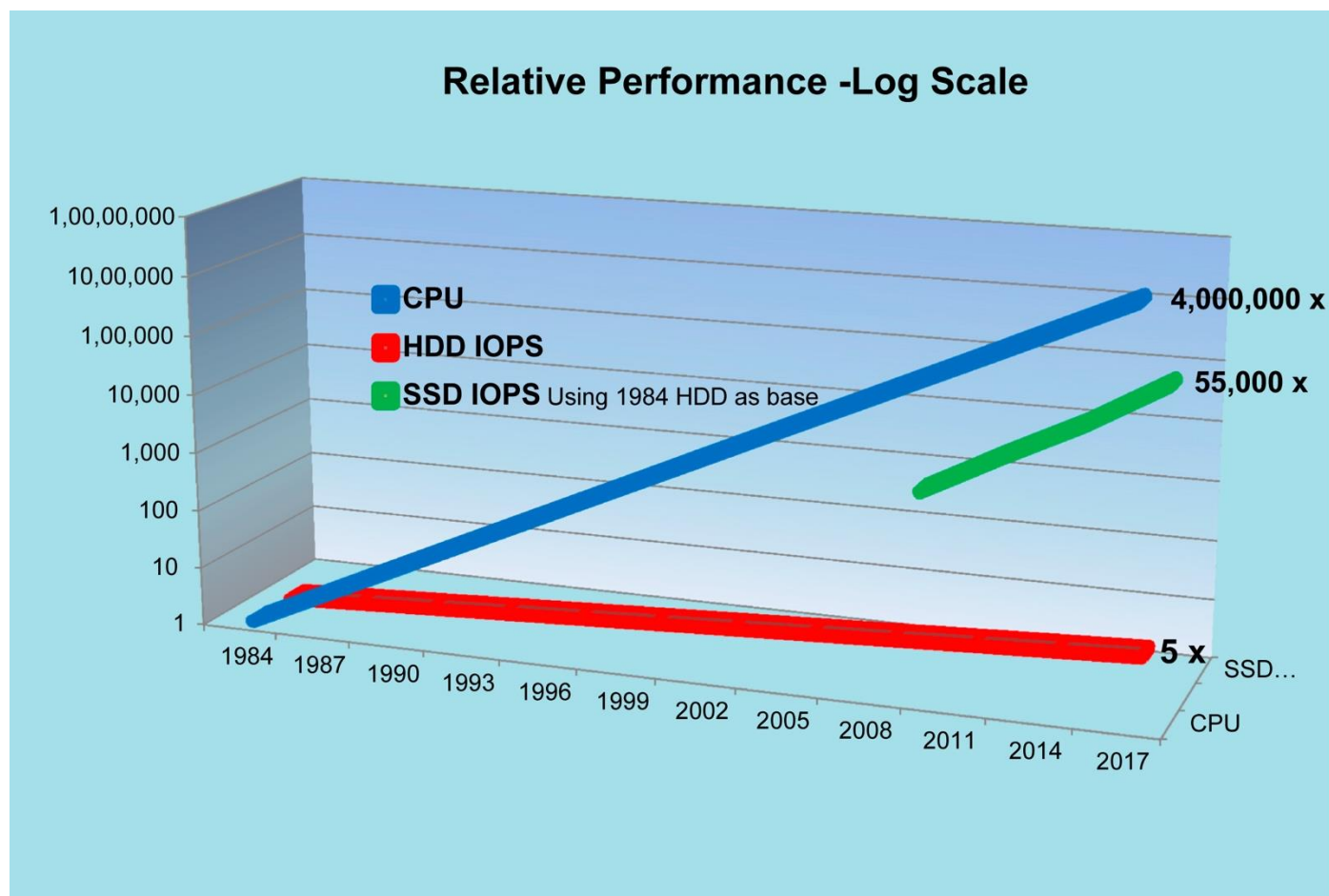


No need to be directly near to computations:

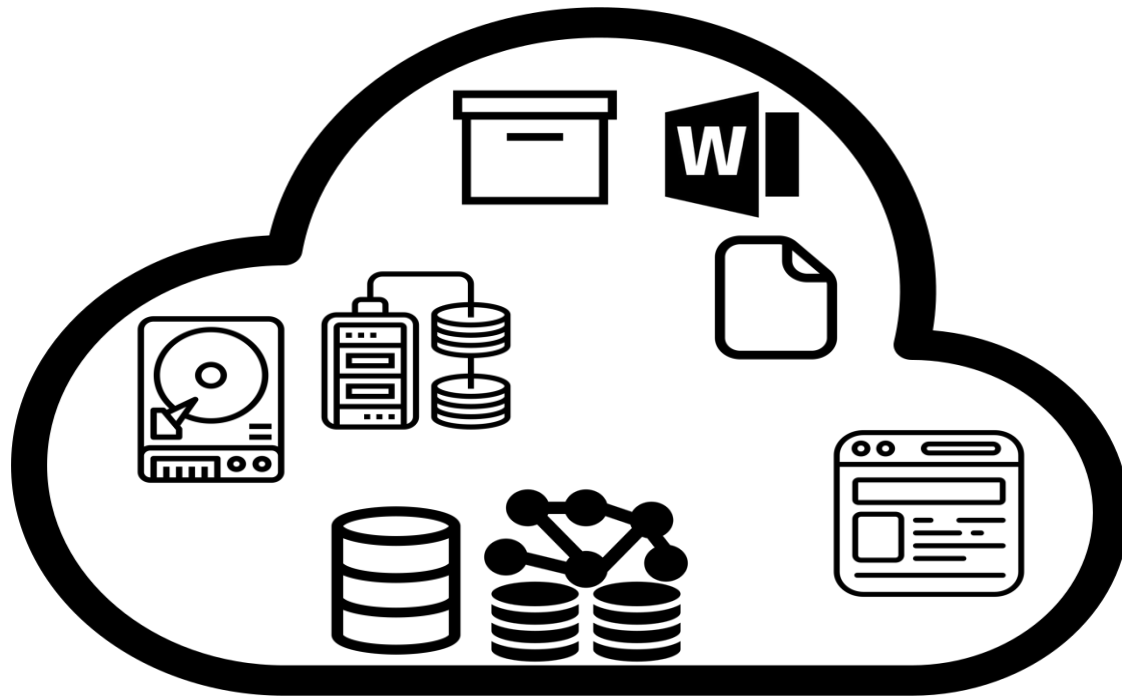
- Bandwidth increases massively
- Architectures tackle latency
- Software organizes access

Why Storage matters?

→ Secondary Storage keeps up with Moores' Laws



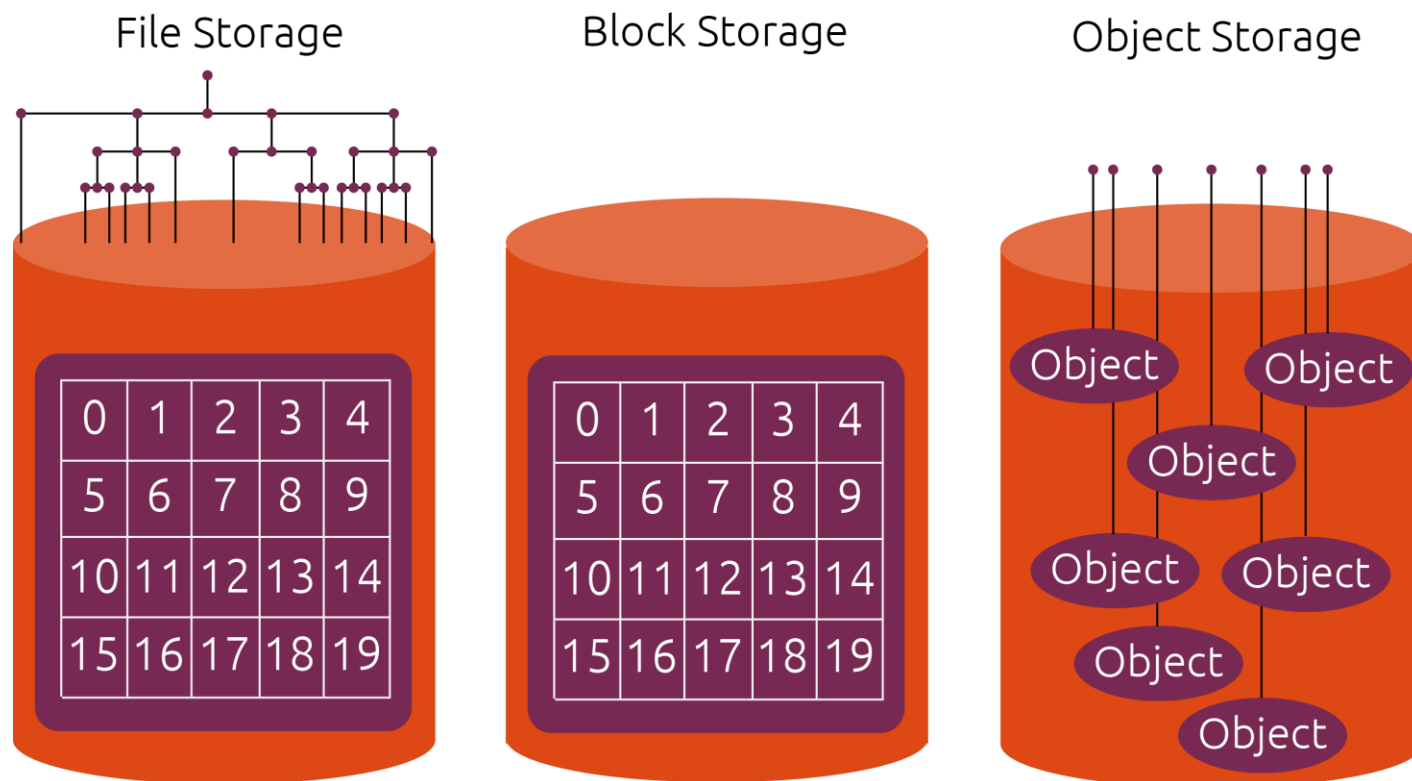
- IOPS of SSD increase the same scale as CPU-cycles
- At the same time, SSD prices become lower than HDD prices for enterprise hardware
- IOPS more important than plain bandwidth



Cloud is Service First

- **Disk-based Storage** → Blocks
- **File-based Storage** → Folders / Buckets / Filesystems
- **Structured Storage** → (No)SQL Datastores
- **Displaying Storage** → Buckets, SaaS-Services of any kind

What is Storage?



Block Storage:

- Addressing storage areas on disk ("blocks")

File Storage:

- Addressing files in a filesystem on disks

Object Storage:

- Addressing "objects", which are technically based on files

What is a Blockstorage?

Blocks are just a fixed sized area of data on the disk.

Unique Identifies of a block are:

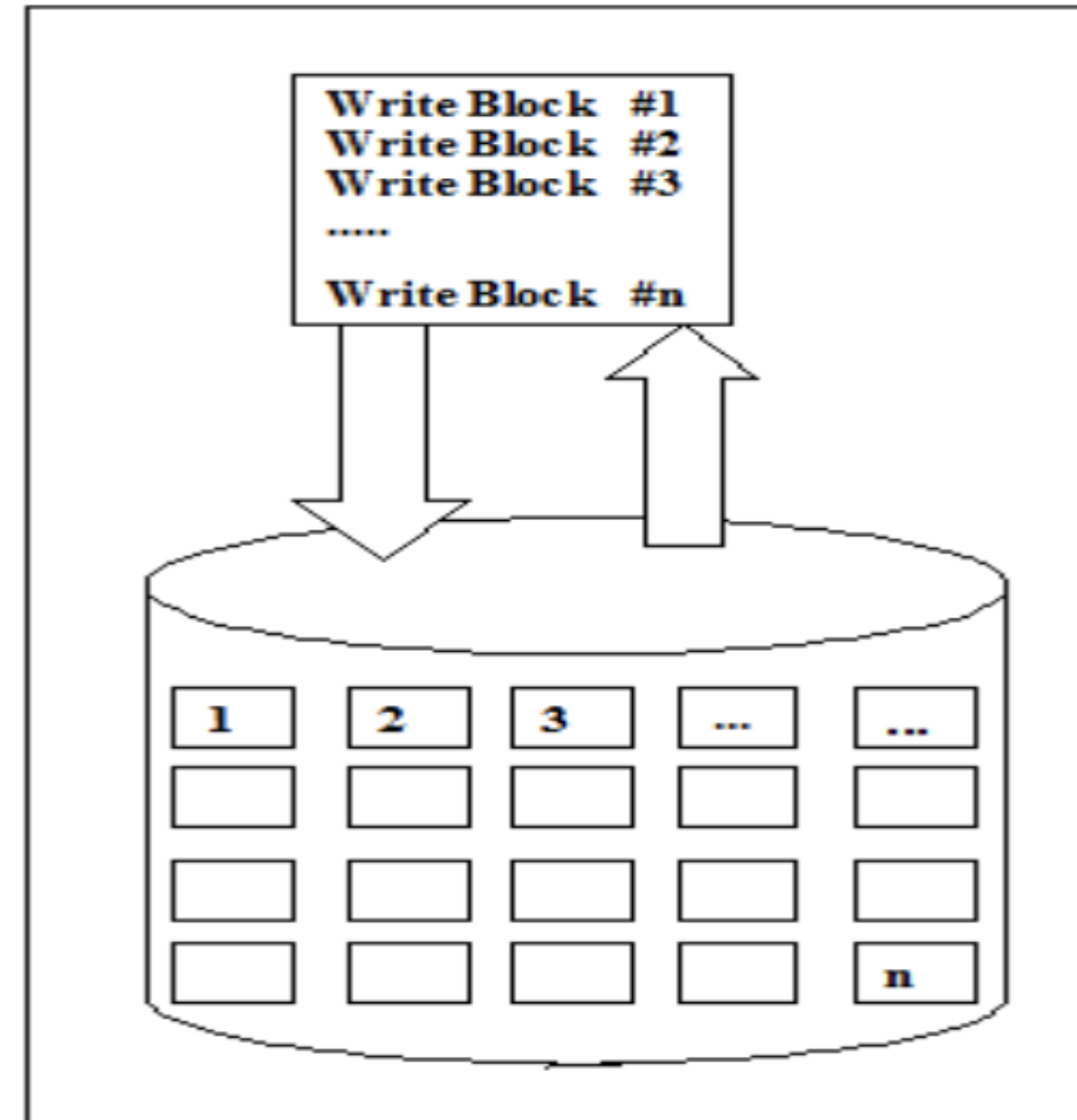
- Offset on the disk
- Length of the block (usually all blocks have the same length)

→ Operated by Operating System

→ No Index / Metadata: Everything is a block

→ Blocks may have pointers to other blocks
(remember Inodes?)

→ Accessing Protocols are iSCSI, FibreChannel,
SATA, NVMe...



Accessing Block Storage, iSCSI

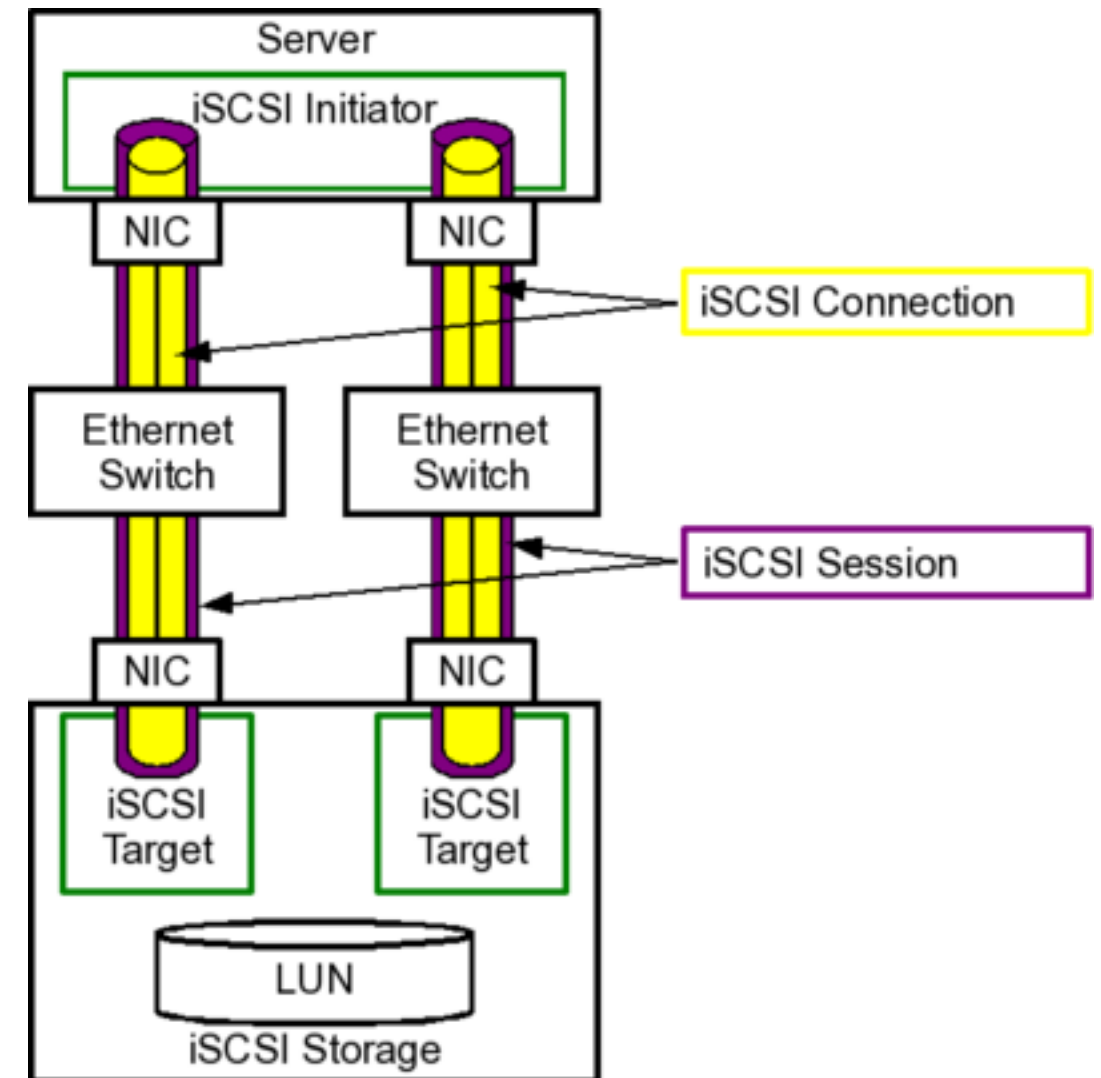
Access to blocks take place via Server/Client Setup and plain TCP/IP (Port 3260):

- Server → iSCSI target
- Client → iSCSI initiator

Target and Initiator can be in Software or Hardware:
→ In Software, they are often part of the OS

To access, a session and a connection must be used:

- Session: permanent setup between initiator and target, used for lookup and init
- Connection: Single access for reading and writing blocks (offset / length) within an existing session



Benefits / Drawbacks of Blockstorage

Benefits:

- Flexibel, eigenes FS
- Einfache Verschlüsselung (symmetrisch)
- Vergleichsweise schnell
(wenig applikatorischen Overhead)

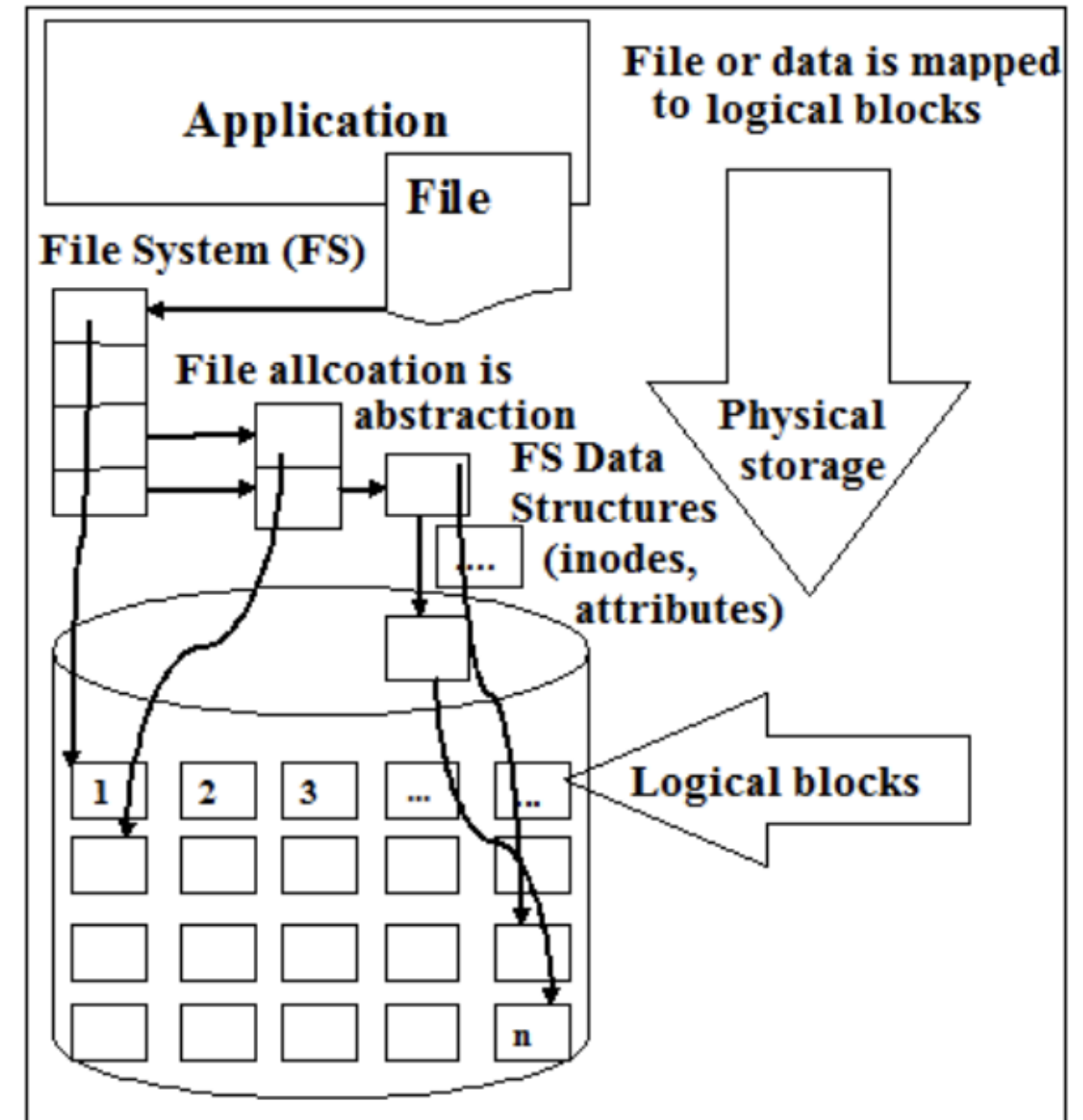
Drawbacks:

- Kein Lesen der gesamten Blockgrösse
- Nicht direkt für (die meisten) Applikation nutzbar
- Keine fein granulare Authorisierung auf einzelne Blöcke

What is a Filestorage?

Files are elements of a filesystem, bringing structure in the chaos of blocks.

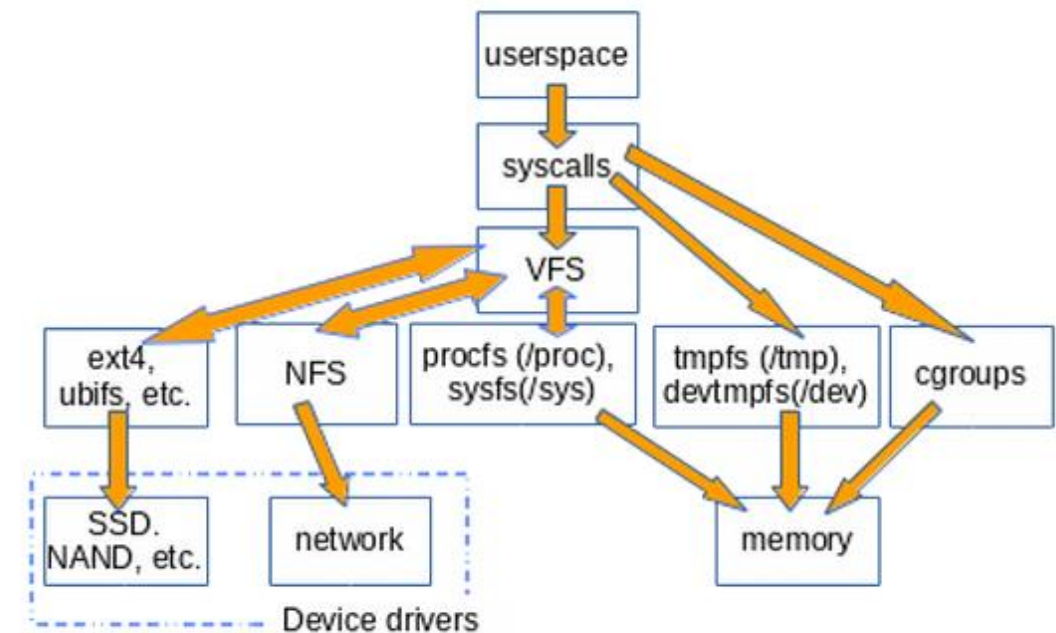
- Overlaying structure of blocks necessary (e.g. INodes in Linux Systems)
- Variable length of elements
- Depending on OS
- Internally hierarchically structured (mapping partially the structure of block pointers)
- Common Filesystems are ext[2-4[, btrfs, ntfs, apfs
- Accessing Protocols are SMB, SSH, NFS, HTTP (Webdav)



Accessing Files / VFS

Filesystems bring order in the list of blocks:
(In fact, they are organizing entire operating systems e.g. with VFS):

- Flexible sized elements (thanks to the Inode Structure)
- Owner / Access Management of Elements
- Links
- Further OS-Management:
 - Processorganization
 - Ressourcemanagement (remember cgroups?)
 - Devicemanagement



Benefits / Drawbacks of Filestorage

Benefits:

- Direkter Zugriff auf Daten von Applikation nötig mit am kompatibelsten für Anwendungen
- Dahinterliegende Logik ist nicht in der Applikation
- Permissions / Rechtegmt / Authorisierung möglich
- Integritätsschutz / Journaling / Snapshots

Drawbacks:

- Jedes Dateisystem hat Flaws
- Overhead im Lesen/Schreiben
 - Einhergehend mit Funktionalität
 - Einhergehend mit der Blockstruktur
- OS-abhängig

What is a Object Store?

- Objects are larger binary data referenced by a unique key:
 - Flat structure, no hierarchy
- Variable length of values
- Depending on OS, used by applications often
- Can be enriched by metadata
- Can be implemented by a filesystem with flat hierarchy as backend
- Recommended Pattern: Write once, read often
- Really becoming popular due to the cloud move since application can easy access data directly most often based on HTTP

ID	Binary Data	Meta Data
10216	101 101 1010101010101010101010101010001010101010100110 10011	Name 1 Value 1 Name 2 Value 2 Name N Value N



Accessing Object Stores / REST

example URL format	GET	POST	DELETE
<object_type>	returns object list (query, filters) 200	creates new object OR makes bulk update 201 / 200	bulk delete (query, filters) 200
<object_type>/<id>/	returns single object, object-specific filters 200	updates single object 200	deletes single object 200
<object_type>/<id>/acl/	returns object permissions 200	updates object permissions 200	Not supported 405

- Request: GET URL/<object>/ [<subobject_id>] / [?<params>]
- Request: DELETE URL/<object>/ [<subobject_id>]
- Request: POST URL/<object>/ [<subobject_id>]

Benefits / Drawbacks of Objectstorage

Benefits:

- “einfach” damit zu arbeiten:
Standardmittel des Webs werden verwenden
- Einfacheres Exponieren der Dateien,
transparentere Authorisierungsmechanismen
- Skalierung

Drawbacks:

- Applikation muss es unterstützen
(analog zu Blockdevice)
- Grosser Overhead bei Lese-/Schreibzugriffe
- Limitierter Anwendungsbereich

SAN, DAS, NAS

DAS:

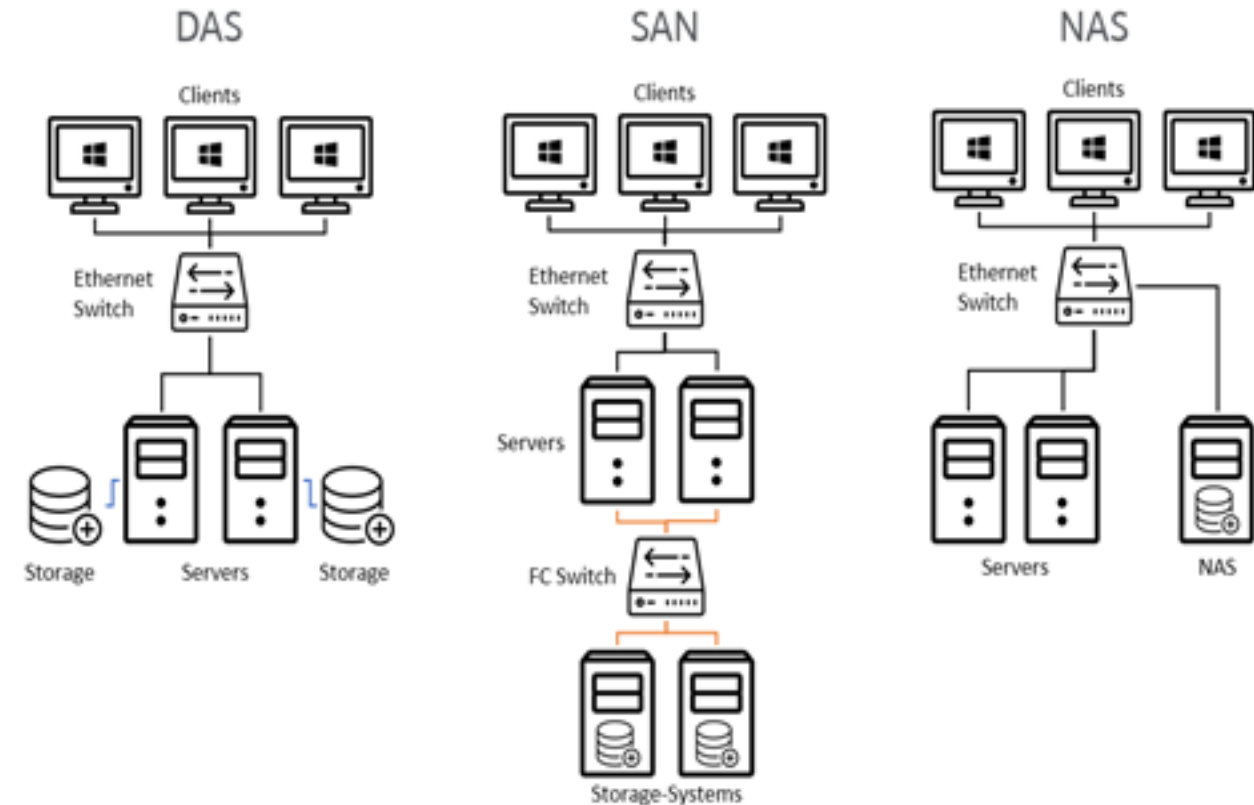
- Disk directly attached to server via SCSI / IDE/ SATA
- Cheap, easy, fast setup
- If server goes down, disk go down

SAN:

- Disk attached via FibreChannel to one/multiple servers
- Complex, second stack, more expensive
- Failover / Scaling possible

NAS:

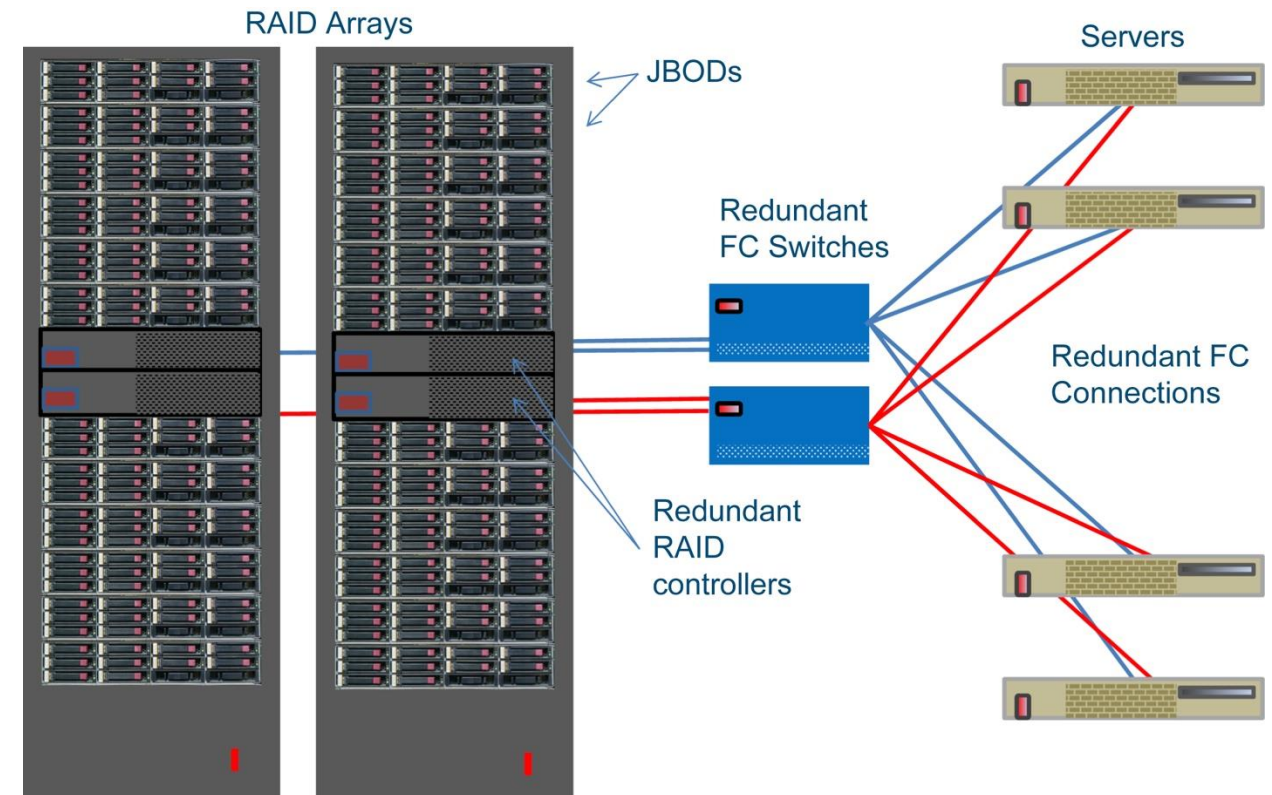
- Disk attached directly as device to network
- Easier technical setup, cheaper, redundant
- Blockaccess needs workaround e.g. iSCSI



Example: Classical SAN

JBOD: Just a bunch of disks

- Arranged in RAIDs
(Redundant Array of Inexpensive Disks)
- Accessible via Fibre Channel Switch
(can be a normal switch when working with iSCSI:
 - RAID-Arrays represent iSCSI targets in that case)

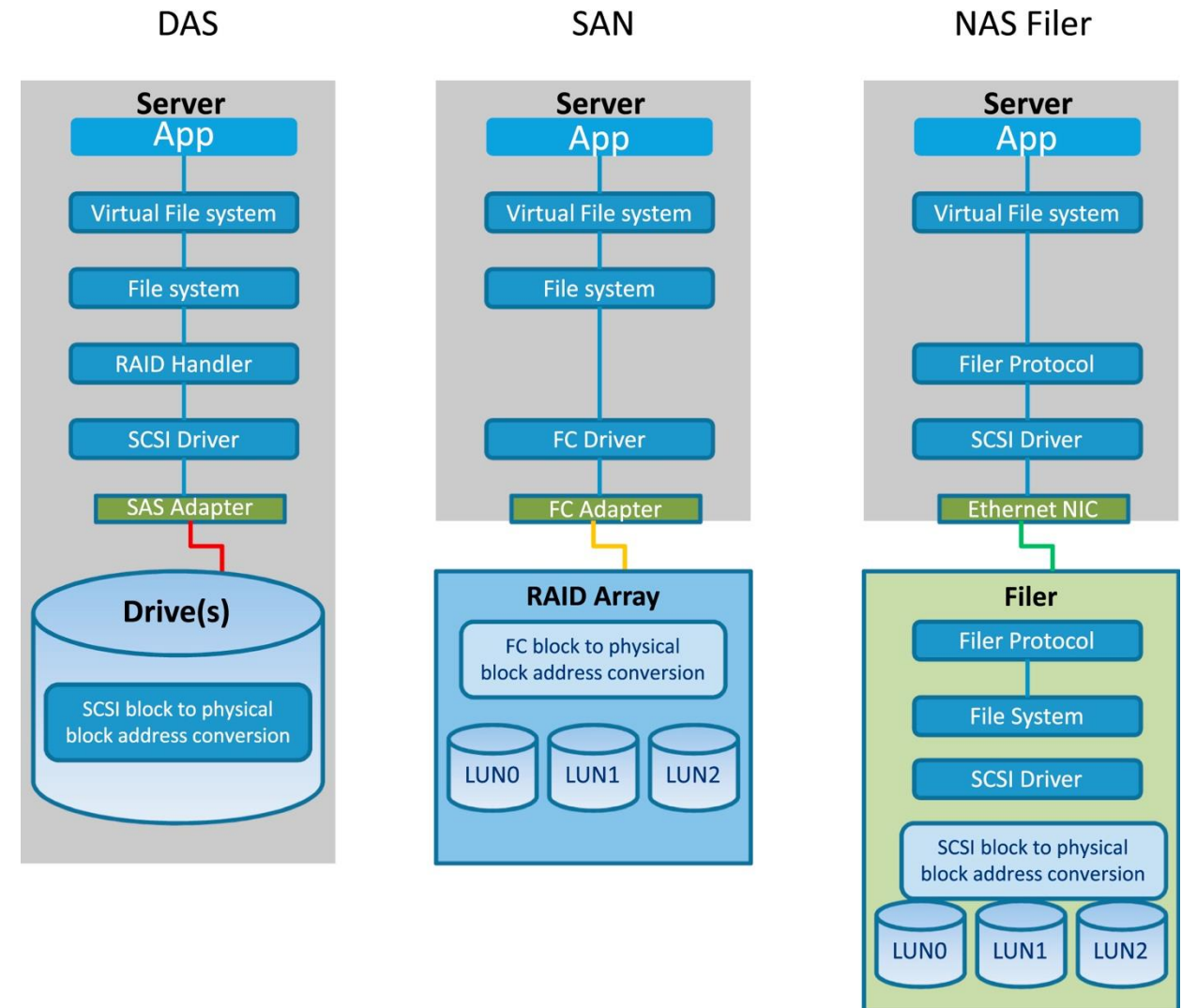


Problem: hardware restriction
(capacity / availability / etc.)

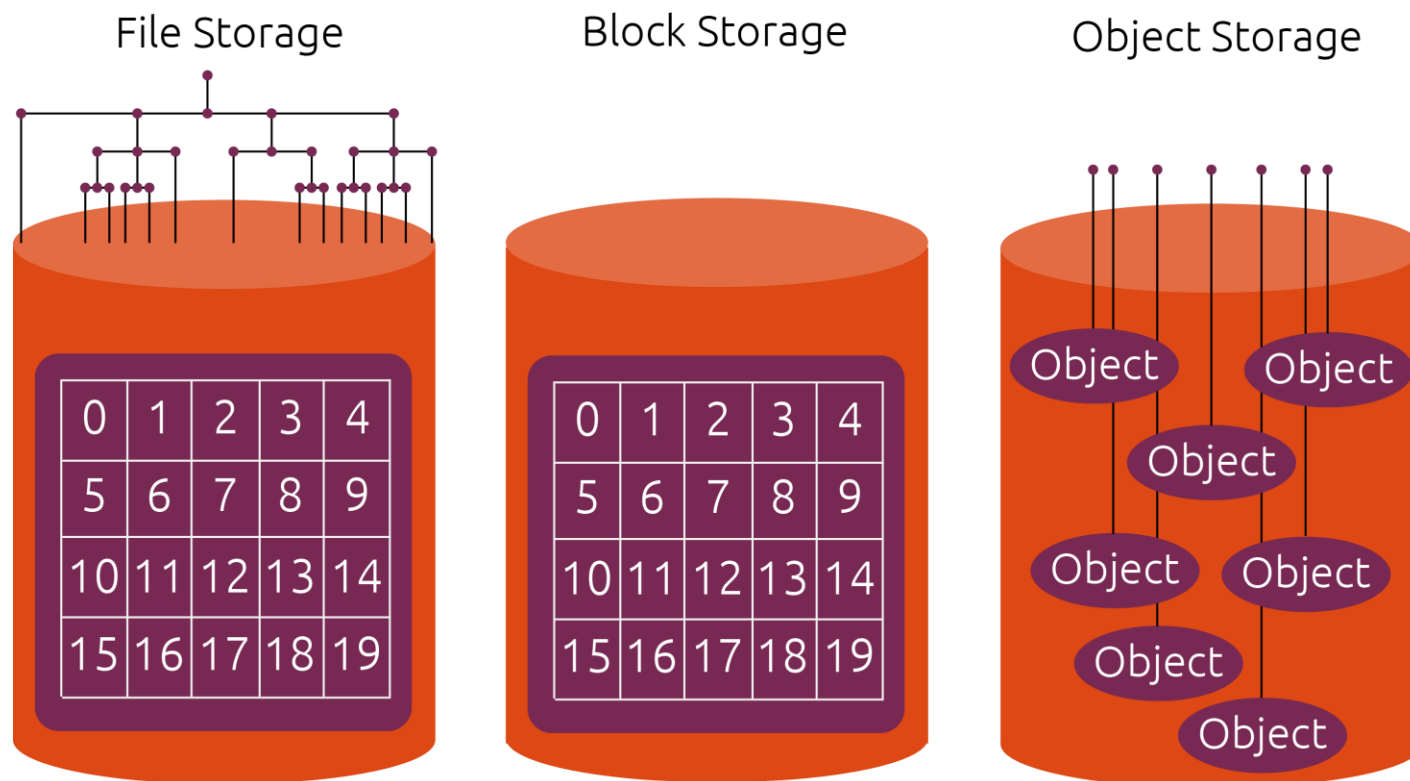
Mapping to storage kind

Level of abstraction so far is Blockstorage

- DAS/SAN: Metadata of file system is on disk themselves
 - (-) iterating over hierarchy needs to traverse the entire TCP/IP-Stack (remember OS-Layers)
- NAS: Metadata is on nodes themselves
 - Iterating through structure is faster
 - (-) Filer / NAS-nodes have own OS/Kernel with all complexity



Reminder: What is Storage?



Block Storage:

- Addressing storage areas on disk ("blocks")

File Storage:

- Addressing files in a filesystem on disks

Object Storage:

- Addressing "objects", which are technically based on files

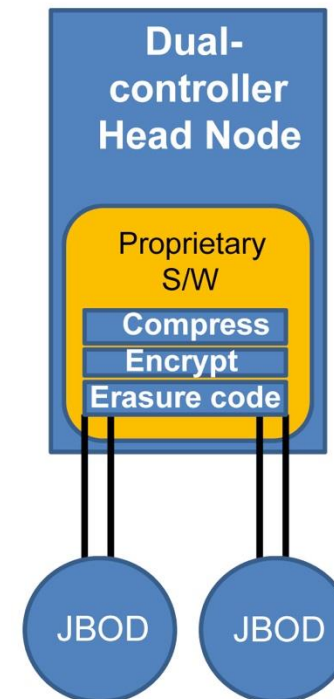
Software defined Storage to the rescue

SDS "virtualizes" the storage

- Storage accesses are stacked before "storage" (compress & erasure / compress & encrypt)
- Access to concrete storage is abstracted
 - Several machines offer software access to storage
 - Storage layer is entirely independent from data to read/write
 - HA/extension is easier

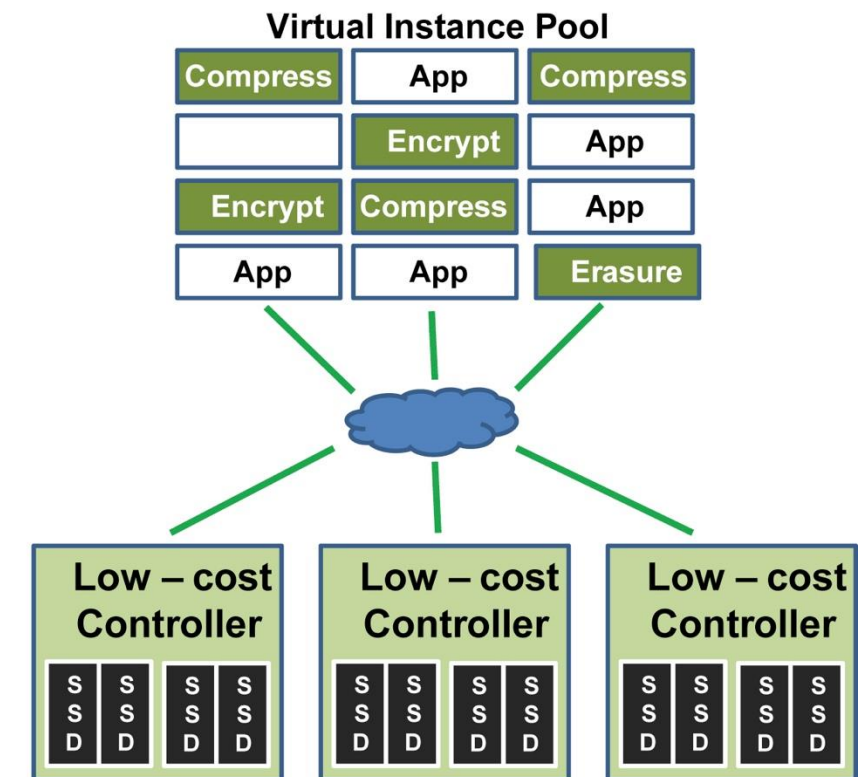
What is the problem?

Traditional Storage



80-drive JBODs

SDS



Compact appliances 10 to 12 drives

SDS-Example in practice: Ceph

Ceph: Scale testing

Bigbang scale tests mutually benefitting CERN Ceph

Bigbang I: 30PB, 7200 OSDs, Ceph Hammer

Found several osdmap limitations

Bigbang II: Similar size, Ceph Jewel

Scalability limited by OSD-MON traffic.

Lead to development of ceph-mgr

Bigbang III: 65PB, 10800 OSDs, Ceph Luminous

No major issue found



20th October 2017

Storage at CERN

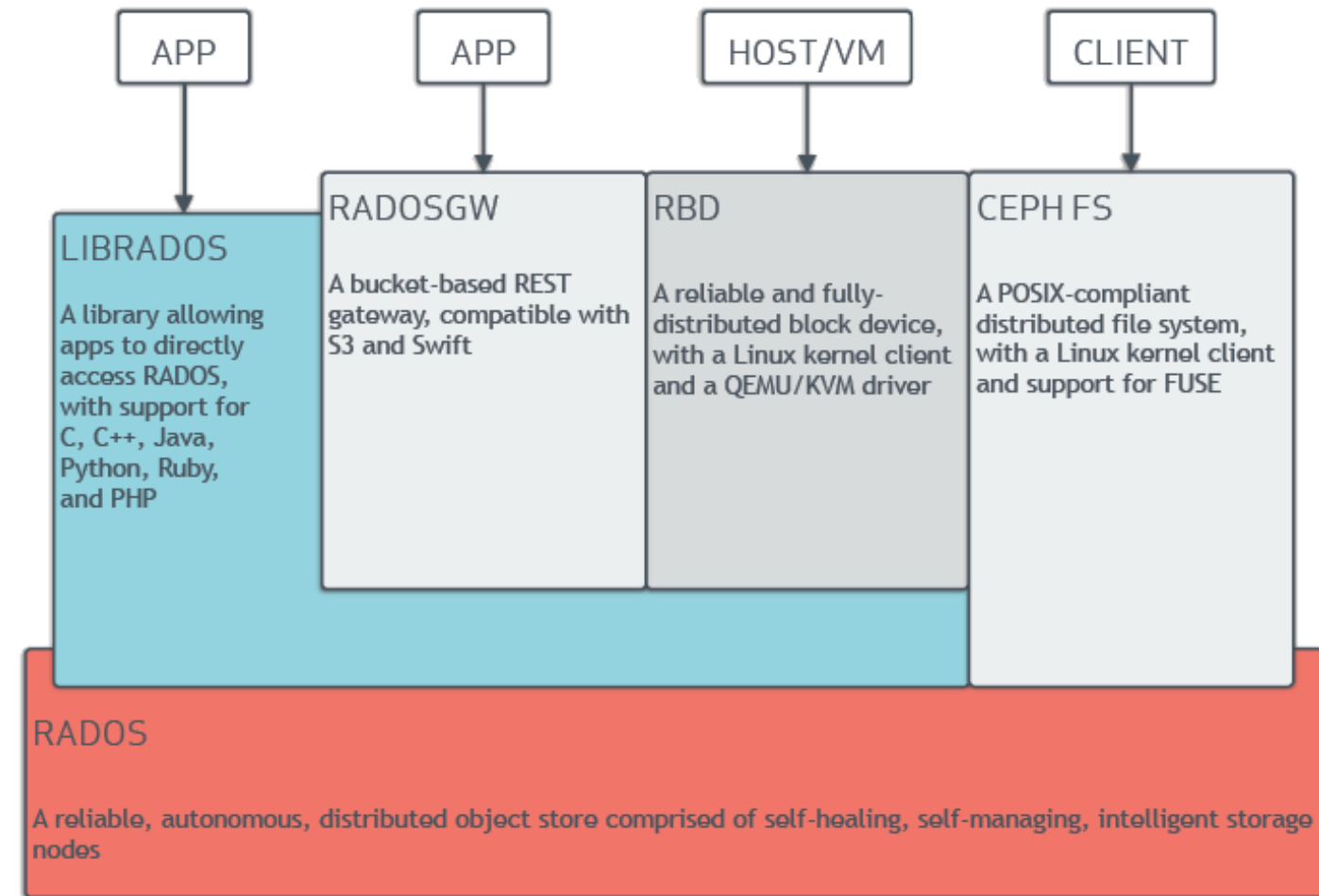
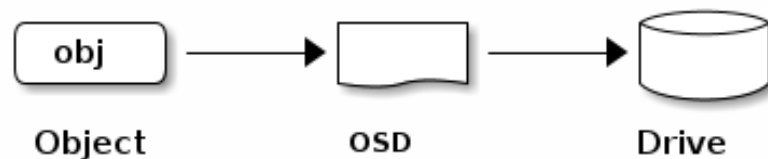
16

<https://indico.cern.ch/event/649159/contributions/2761965/attachments/1544385/2423339/hroussea-storage-at-CERN.pdf>

Concrete Example of SDS, CEPH

Offering interfaces for:

- Block: RBD = iSCSI target
 - File: CephFS as Filesystem
 - Object: RadosGW as Bucketstorage
- All data is transferred to a OSD (Open Stack Demon) mapping drives on blocklevel



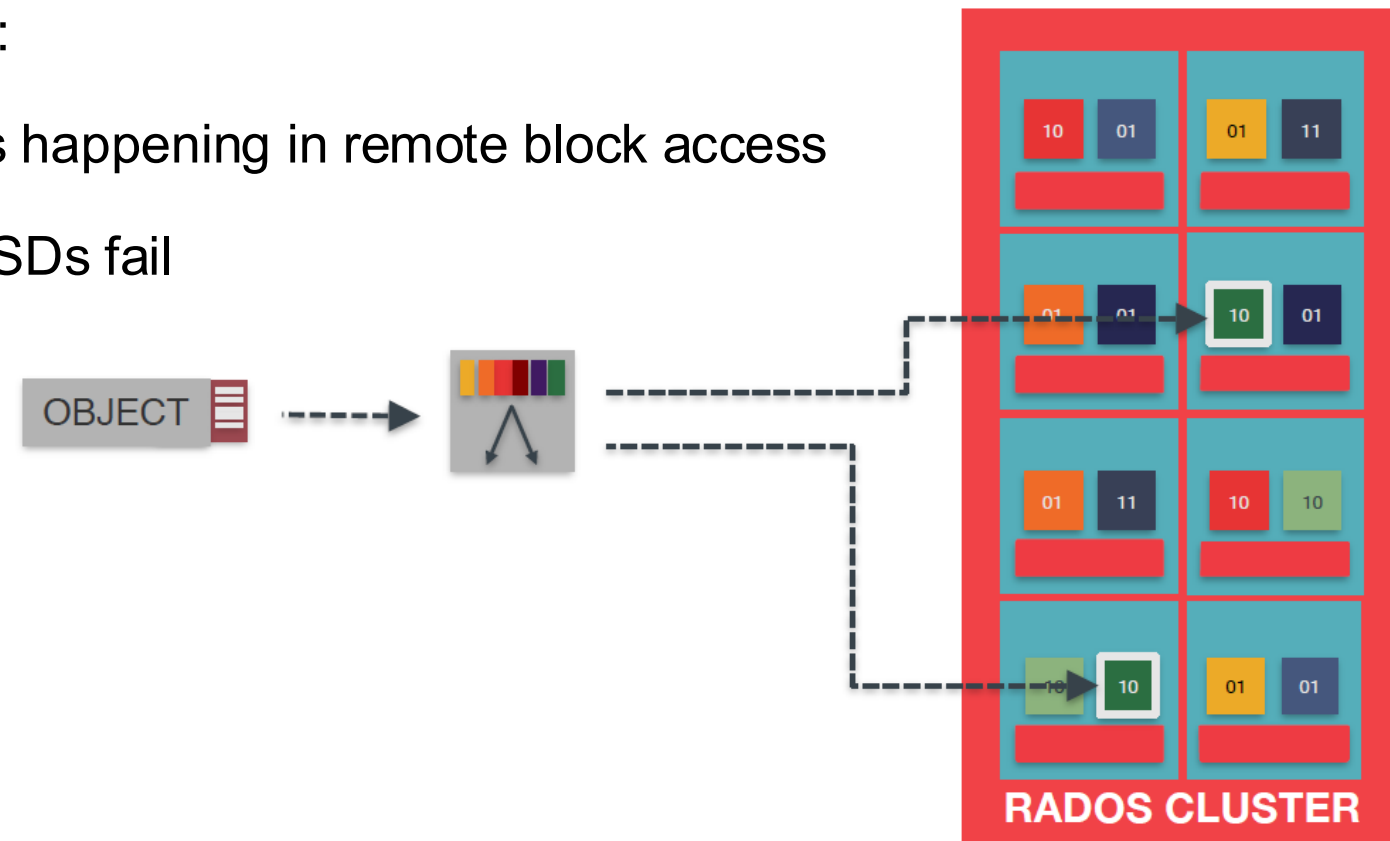
OSDs

OSDs are aware of other OSDs and cluster status:

- overcoming the intense lookup over Ethernet as happening in remote block access
- helping with HA since data is replicated to let OSDs fail

Further features include:

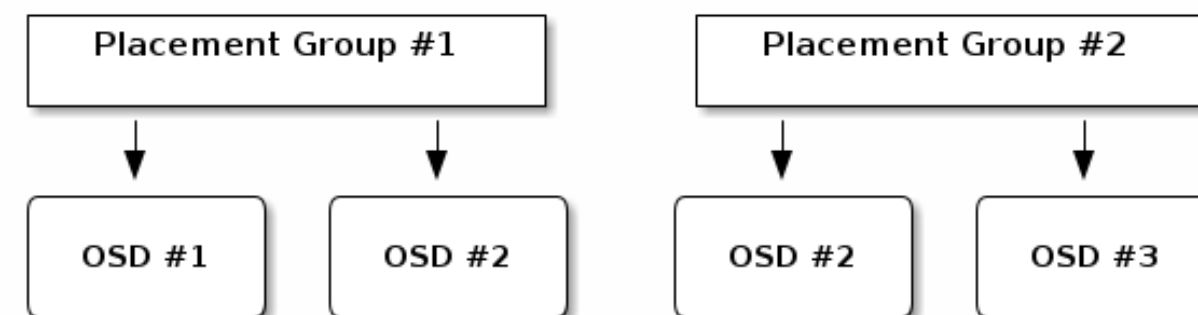
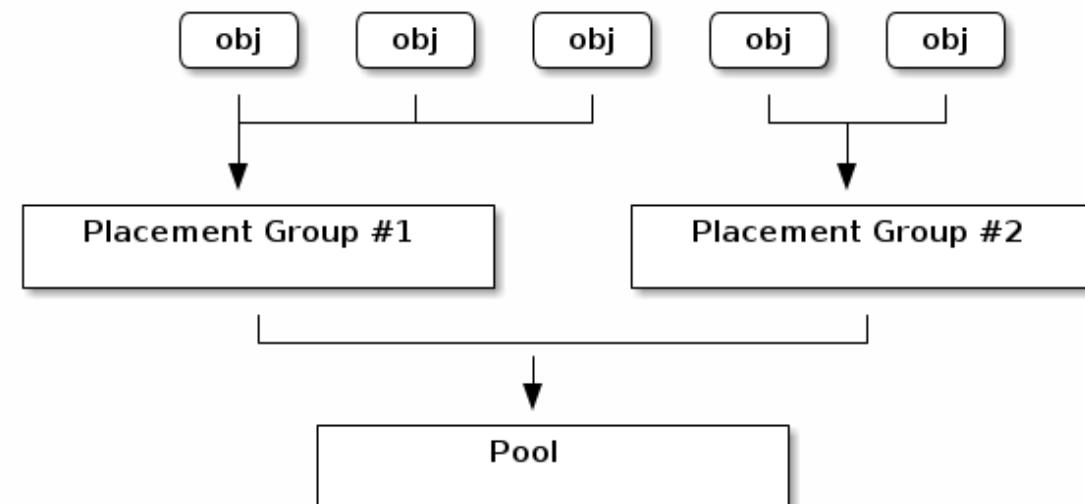
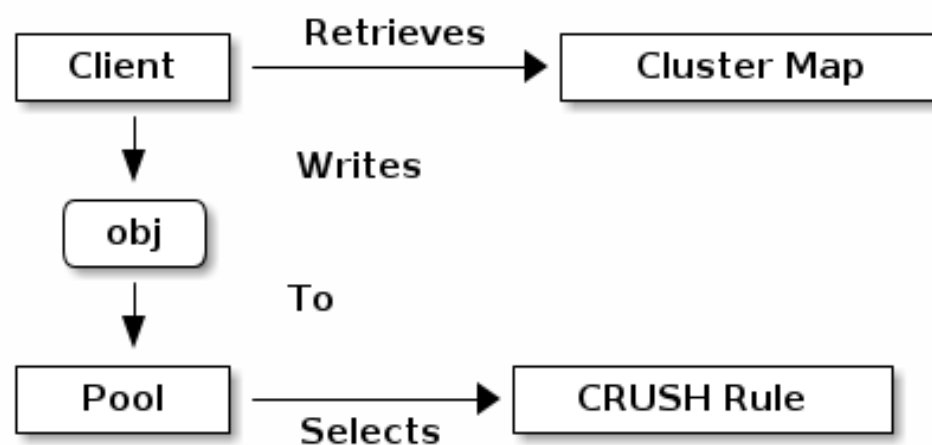
- Direct connect to OSD-clients
- Status of OSD is broadcasted
- Data is scrubbed / verified at all time
- Data is replicated to several sites



Mapping Data to OSDs

Placement Groups:

- Combine several OSDs
- Can be part of a larger tree/DAG-structure building up pools



CRUSH-Algorithm

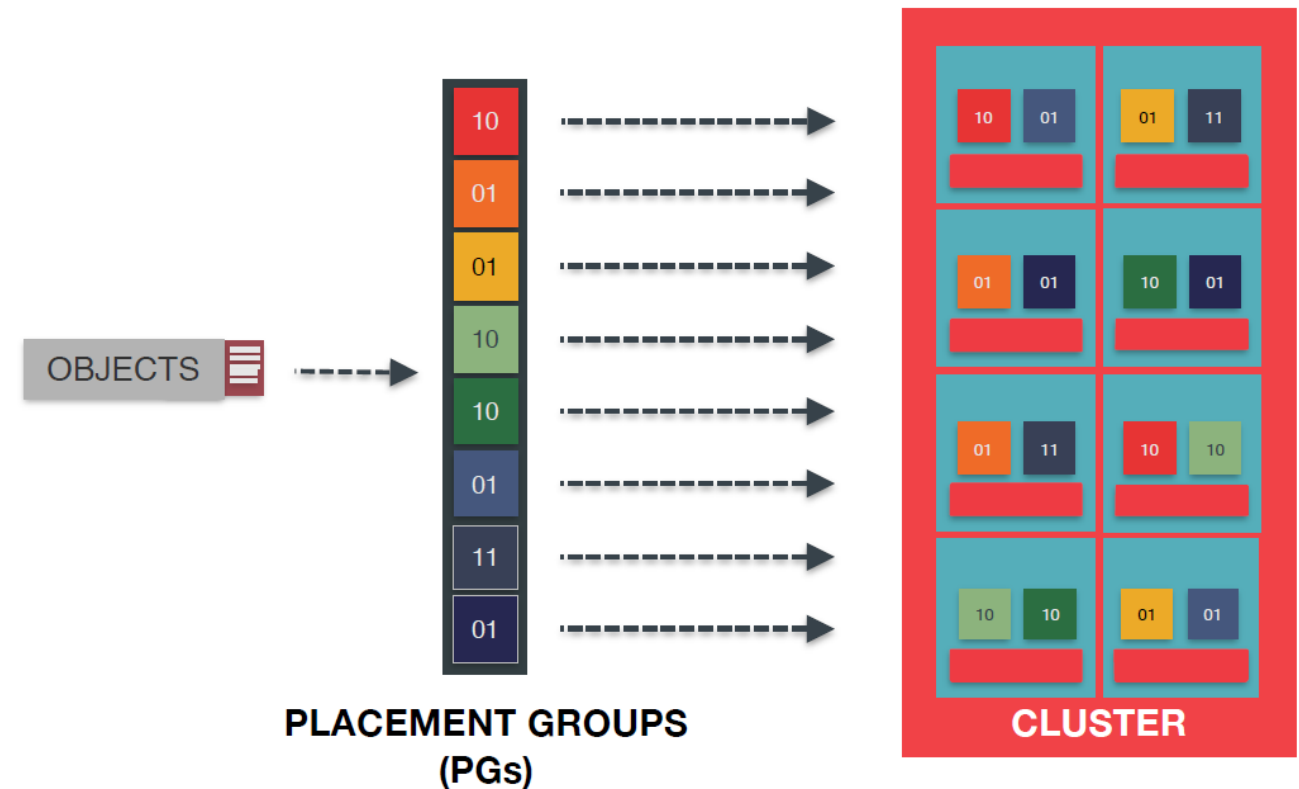
Based on:

- Clustermap, which is decentralized / versioned
- Local computation of hashes / lookups

No centralized continuous lookup necessary!

Request to write object resulting in:

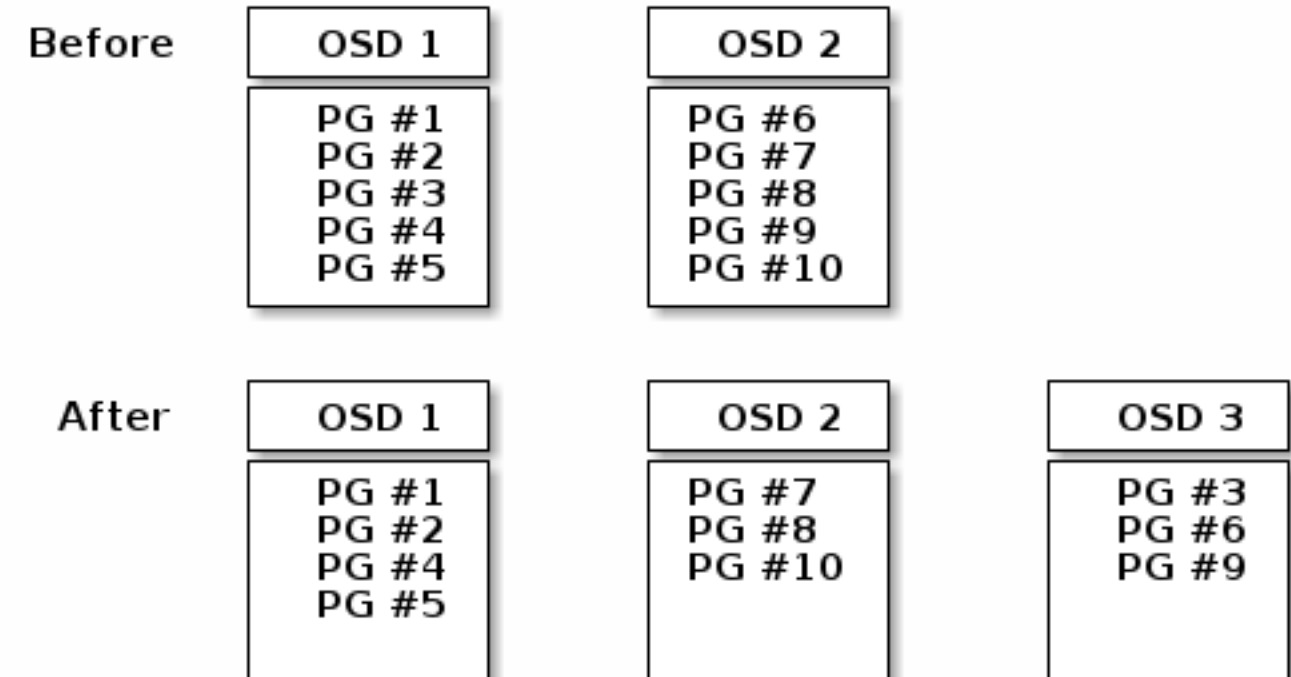
1. Hashing of object id
2. Lookup in cluster map to determine placement group
3. Lookup in placementgroup to determine OSDs



Modifying Disks

When OSDs are altered, rebalancing takes place:

- The Clustermap is altered and updated
- Based on Object ID, the new destination is computed according to the CRUSH-algorithm and the new Clustermap
- Data on Rest is transferred to the new OSD



CSI and K8s

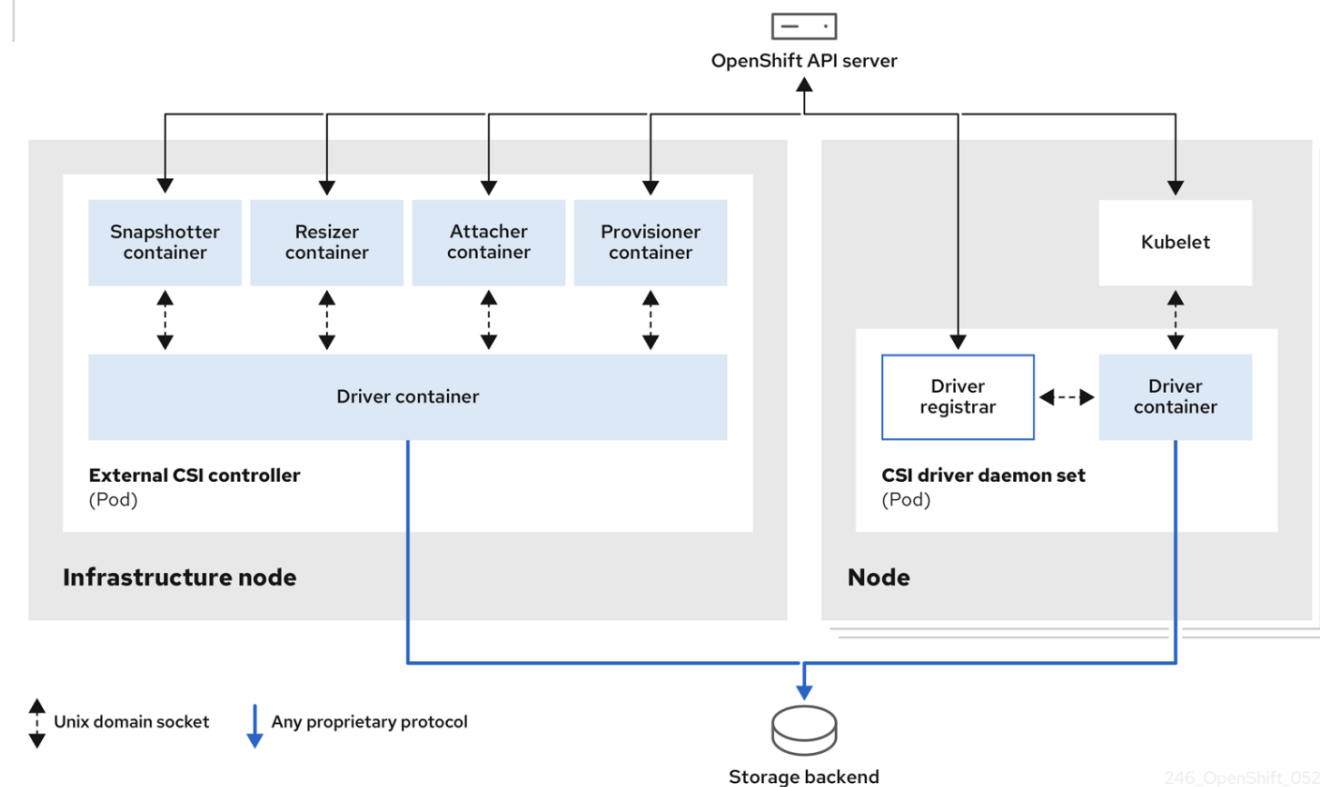
CSI abstracts the usage of proprietary storages in any Kubernetes Cluster offering:

- PVCs
- StorageClasses

Users of Kubernetes can get storage via PVC as:

- File Storage
- Block Storage

per default



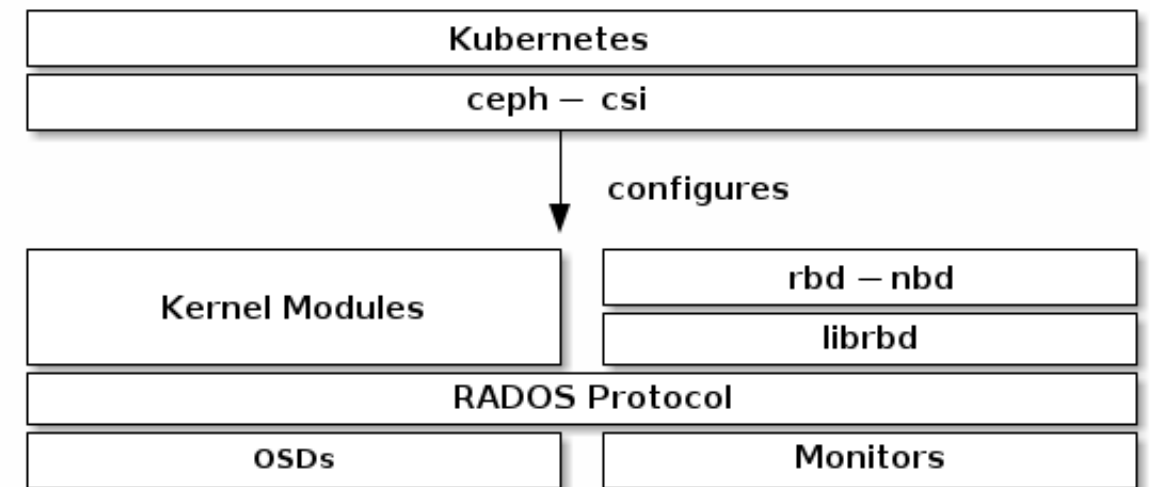
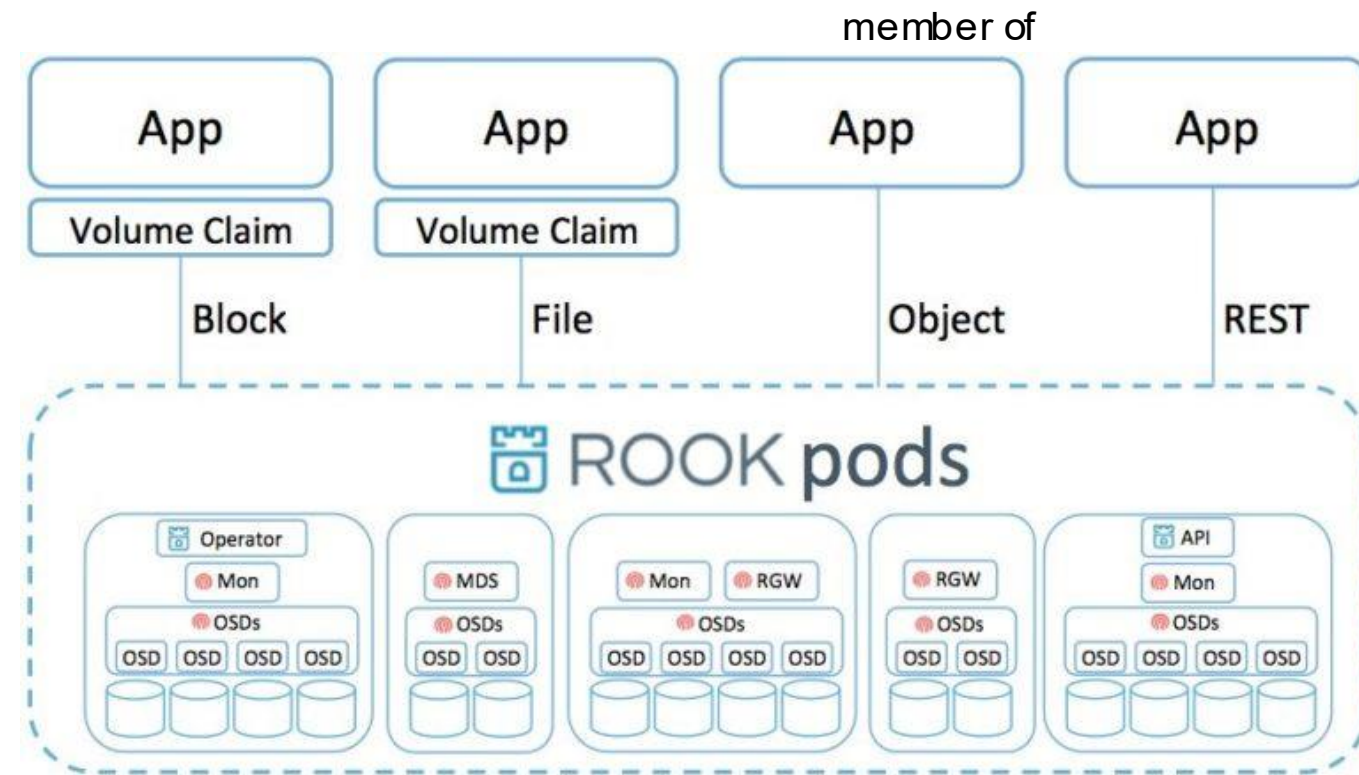
<https://kubernetes-csi.github.io/docs/drivers.html>

CephFS	cephfs.csi.ceph.com	v0.3, ≥v1.0.0	A Container Storage Interface (CSI) Driver for CephFS	Persistent
Ceph RBD	rbd.csi.ceph.com	v0.3, ≥v1.0.0	A Container Storage Interface (CSI) Driver for Ceph RBD	Persistent

Ceph and K8s → Rook

Ceph can also be deployed in Kubernetes abstracting / aligning the underlying storage:

- All elements of Ceph are deployed as pods and maintained by an operator
- The “backend” is configured via `CephCluster`
- For the “frontend”, all three interfaces are available: Block, File and Objects



Summary

- Storage Platforms switch to virtualized platforms as well
 - Higher availability
 - More scalability
- Dedicated Storagelayers for dedicated use cases
 - Block Storage
 - File Storage
 - Object Storage

However, borders are blurring

(see Kubernetes StorageClass-Kinds)

- Platforms can be platforms for platform which can be platforms...

Summarizing Questions

- What are typical use cases of a block storage?
- What protocols work with object storages?
- What is the benefit of using software defined storage such as Ceph?
- What are the mechanisms that make Ceph resilient against failing hardware?
- What is the purpose of an OSD in Ceph?